Aula 14

Análise de Algoritmos

Enumeração e Força Bruta
(Parte 1)

- Em alguns problemas, a única estratégia que conhecemos consiste em enumerar todas as possíveis soluções e testar qual é a melhor
- Em alguns casos, é possível eliminar algumas soluções sem ter que enumerar todas
- Algoritmos baseados nessa ideia são chamados de algoritmos de **força bruta**
- A maioria desses algoritmos não é eficiente pois é comum que o número de possíveis soluções seja exponencial, mas são eficientes em alguns casos

- Vamos começar com um problema simples, para o qual o algoritmo por força bruta é eficiente

Problema: Triplas de Múltiplos

Dado um vetor de inteiros positivos A[1..n], contar o número de triplas

(i, j, k) com i < j < k tais que

A[i] divide A[j] e A[j] divide A[k].

A[j] % A[i] = 0

A[k] % A[j] = 0

<u>Exemplo</u>: A = [1, 2, 3, 4, 8]
1 2 3 4 5

Triplas válidas: (1,2,4), (1,2,5), (1,4,5), (2,4,5)
Resposta: 4

- A solução mais direta para esse problema é verificar cada tripla (i,j,k) em laços aninhados
- Não é claro que exista uma estratégia melhor do que essa, pois não há uma estrutura óbvia para ser explorada

```
count = 0
para i = 1 até n:
  para j = i + 1 até n:  → próximo j
    se A[j] % A[i] == 0:
      para k = j + 1 até n:  → próximo k
        se A[k] % A[j] == 0:
          count = count + 1
retorne count
```

- Claramente, este algoritmo tem tempo $\Theta(n^3)$ e é eficiente
- A facilidade deste código é que precisamos procurar apenas 3 elementos

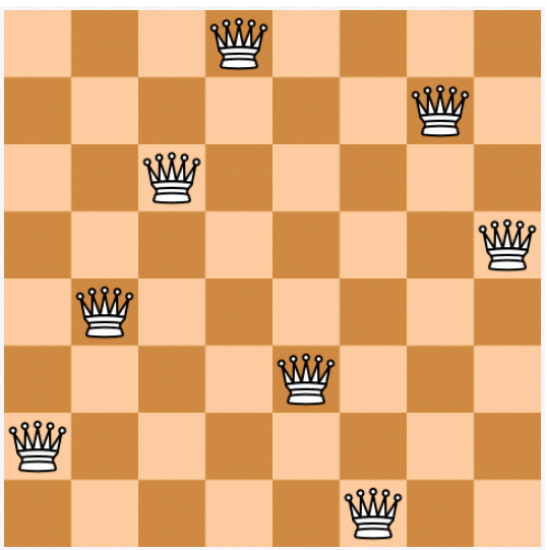
Lista 14A - Q1

Se quisermos generalizar o algoritmo de triplas para g-tuplas, qual seria o tempo de execução?

Qual o número de g-tuplas que precisam ser analisadas?

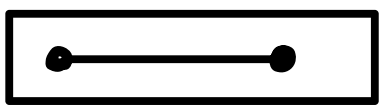
Um problema famoso de força bruta é o problema das rainhas:

Problema das rainhas:
Posicionar 8 rainhas em um tabuleiro 8×8 de modo que nenhuma rainha ataque outra.
Contar todas as soluções.



Vamos ver para 4 rainhas em 4x4

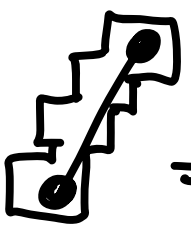
```python
def atacada(linha1,coluna1,linha2,coluna2):
    if (linha1 == linha2 or
        coluna1 == coluna2 or
        linha1-linha2 == coluna1-coluna2 or
        linha1-linha2 == -coluna1+coluna2):
        return True
    return False
```



linha1 = linha2

coluna1 = coluna2

linha1 - linha2
= coluna1 - coluna2



linha1 - linha2
= - coluna1 + coluna2

Ideia:

- Colocar rainha da linha 1 na coluna $c_1 \in \{1, 2, 3\ 4\}$

- Colocar rainha da linha 2 na coluna $c_2 \in \{1, 2, 3\ 4\}$. De modo que $(1, c_1)$ não é atacada por $(2, c_2)$

- Colocar rainha da linha 3 na coluna $c_3 \in \{1, 2, 3, 4\}$. De modo que $(1, c_1)$ e $(2, c_2)$ não são atacadas por $(3, c_3)$

- Colocar rainha da linha 2 na coluna $c_2 \in \{1, 2, 3\ 4\}$. De modo que $(1, c_1)$, $(2, c_2)$ e $(3, c_3)$ não são atacadas por $(4, c_4)$

4 rainhas em um tabuleiro 4x4:

```
for c1 in range(1,5):
  for c2 in range(1,5):
    if not atacada(1,c1,2,c2):
      for c3 in range(1,5):
        if (not atacada(1,c1,3,c3) and
            not atacada(2,c2,3,c3)):
          for c4 in range(1,5):
            if (not atacada(1,c1,4,c4) and
                not atacada(2,c2,4,c4) and
                not atacada(3,c3,4,c4)):
              print((1,c1),
                    (2,c2),
                    (3,c3),
                    (4,c4))
```

não ataca 1

não ataca 1 e 2

não ataca 1, 2, 3

Lista 14A - Q2

O que acontece se tentarmos colocar 2 rainhas em um tabuleiro 2x2?

E 3 rainhas em um tabuleiro 3x3?

- Naturalmente, podemos resolver o problema das 8 rainhas usando 8 laços encaixados... mas fica feio...

- Mas como poderíamos fazer para n-rainhas em um tabuleiro n por n? Nesse caso, o número de laços encaixados não é fixo...

- Vamos ver uma versão recursiva que é bem elegante

```
n = 8
pos=[0]*(n+1)
count = 0
```

variáveis globais

```
def pode(linha, coluna):
  for i in range(1,linha):
    if atacada(i,pos[i],linha,coluna):
      return False
  return True
```

verifica se podemos colocar uma rainha em (linha, coluna) dado que as outras rainhas estão em (1, pos[1]), (2, pos[2]), ..., (linha-1, pos[linha]-1)

linha a ser preenchida

```
def coloque_rainha(linha):
  global count #because Python is stupid
  if linha == n+1:
     count = count+1
  else:
    for coluna in range(1,n+1):
      if pode(linha,coluna): # pode colocar
        pos[linha]=coluna # coloca
        coloque_rainha(linha+1) # próxima
        pos[linha]=0 # descoloca

coloque_rainha(1)
print(count)
```

colocamos n rainhas

- Será que só é possível fazer com recursão?
- Tudo o que é feito com recursão pode ser escrito sem usar recursão. A pergunta é o quão doloroso isso vai ser

```
n = 8
pos=[0]*(n+1)
count = 0

coloque_rainhas()
print(count)
```

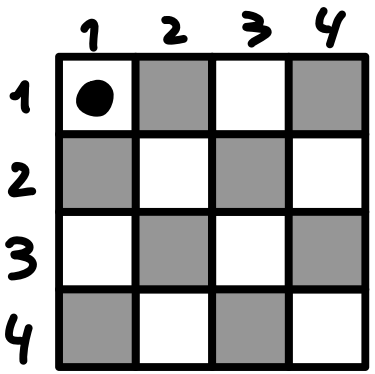
globais

```
def coloque_rainhas():
  global count #because Python is stupid
  linha = 1  — linha a ser preenchida
  while linha>0:
    found = False
    for coluna in range(pos[linha]+1,n+1):
      if pode(linha,coluna):
        pos[linha]=coluna
        linha = linha+1
        if linha == n+1:
          count = count+1
          linha = linha-1
        found = True
        break
    if not found:
      pos[linha]=0
      linha = linha-1
```

n = 4
linha = 1
pos = [0,0,0,0]

linha = 1 > 0

coluna = 1
pode(1,1) ✓
pos [1,0,0,0]
found = True
linha = 2

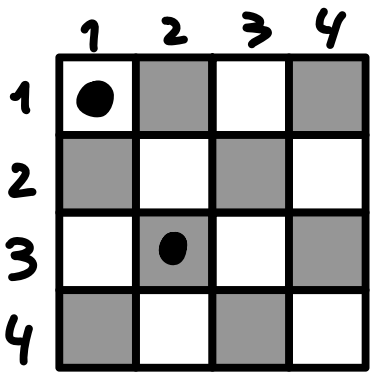


linha = 2 > 0

coluna = 1
pode (2,1) ✗

coluna = 2
pode (2,2) ✗

coluna = 3
Pode (2,3) ✓
pos [1,3,0,0]
linha = 3
found = True



linha = 3 > 0

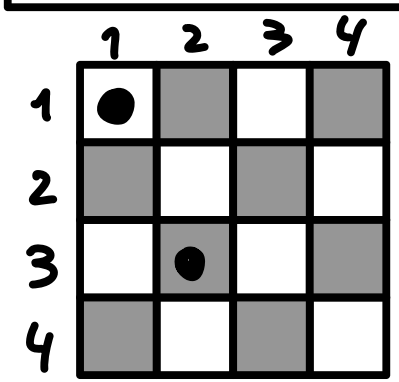
coluna = 1
pode (3,1) ✗

coluna = 2
pode (3,2) ✗

coluna = 3
Pode (3,3) ✗

coluna = 4
pode (3,4) ✗

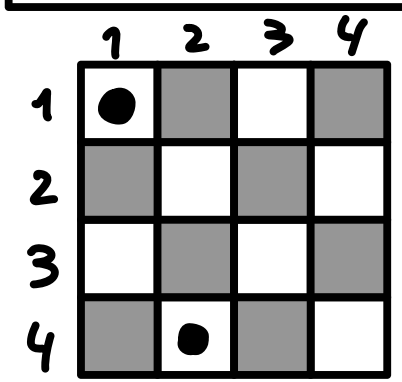
found = False
pos = [1,3,0,0]
linha = 2



linha = 2 > 0

coluna = 4
pode (2,4) ✓

pos [1,4,0,0]
linha = 3
found = True

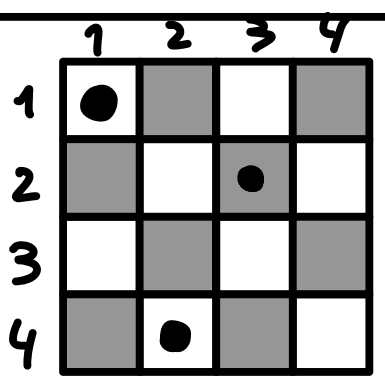


linha = 3 > 0

coluna = 1
pode (3,1) ✗

coluna = 2
pode (3,2) ✓

pos [1,4,2,0]
linha = 4
found = True



linha = 4 > 0

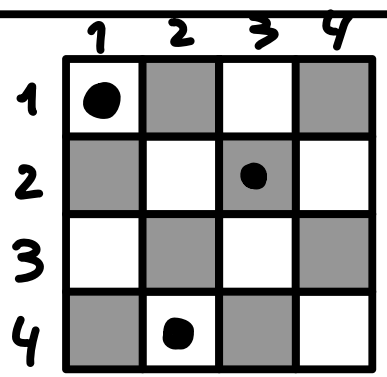
coluna = 1
pode (4,1) ✗

coluna = 2
pode (4,2) ✗

coluna = 3
pode (4,3) ✗

coluna = 4
pode (4,4) ✗

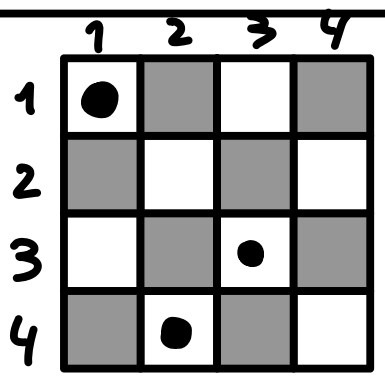
found = False
pos = [1,4,2,0]
linha = 3



linha = 3 > 0

coluna = 3
pode (3,3) ✓

pos [1,4,3,0]
linha = 4
found = True



linha = 4 > 0

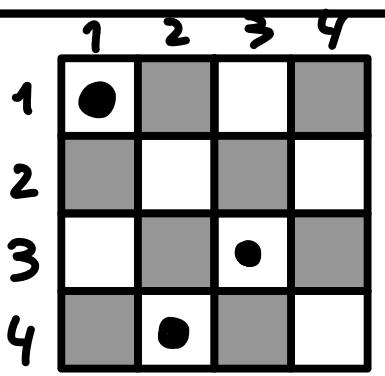
coluna = 1
pode (4,1) ✗

coluna = 2
pode (4,2) ✗

coluna = 3
pode (4,3) ✗

coluna = 4
pode (4,4) ✗

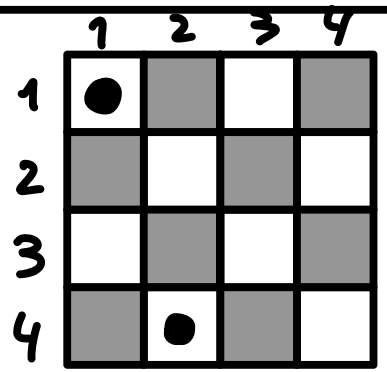
found = False
pos = [1,4,3,0]
linha = 3



linha = 3 > 0

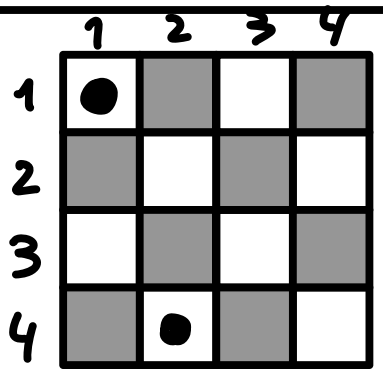
coluna = 4
pode (3, 3) X

found = False
pos [1,4,0,0]
linha = 2



linha = 2 > 0

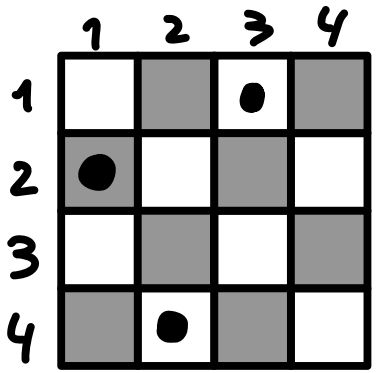
pos [1,0,0,0]
found = False
linha = 1



...

Lista 14A - Q3

linha=4



pos=[2,4,1,0]

linha=4 >0

coluna=1
pode (4,1) a

coluna=2
pode (4,2) b

coluna=3
pode (4,3) c

pos [2,4,1,3]
linha=5
linha=4
found=True

linha=4 >0

coluna=1
pode (4,4) d

pos [2,4,1,e]
found=False
linha=f

Problema: k-Tuplas de Múltiplos

Dado um vetor A[1..n] de inteiros positivos e um inteiro k, contar o número de k-tuplas de índices $(i_1, i_2, ..., i_k)$ com $i_1 < i_2 < ... < i_k$ tais que cada elemento divide o seguinte.

Este problema é uma generalização do primeiro problema, onde k era 3

```
k=3, n=5
A = [0,1,2,3,4,8] #0 is a dummy
pos=[0]*(k+1)
def ktuplas(p):
  if p == k + 1:
    print(pos[1:])
  elif p==1:
    for i in range(1,n+1):
      pos[p] = i
      ktuplas(p + 1)
  else:
    anterior = pos[p - 1]
    for i in range(anterior + 1,n+1):
      if A[i] % A[anterior] == 0:
        pos[p] = i
        ktuplas(p + 1)

ktuplas(1)
```

entrada

global

encontramos k

1ª posição pode ser qualquer

procura próximo

$\Theta(n^k)$

Lista 14A - Q4

Modifique o código para k-tuplas para que imprima $(i_1, \ldots, i_k)$ tal que

$A[i_1] < A[i_2] < \ldots < A[i_k]$.

Qual a primeira k-tupla encontrada para:

$$A = [\underset{1}{7}, \underset{2}{1}, \underset{3}{8}, \underset{4}{3}, \underset{5}{2}, \underset{6}{5}] \quad \text{para } k = 3?$$

O problema das k-tuplas de múltiplos claramente é relacionada ao problema de gerar todos os subconjuntos de tamanho de um conjunto.

Problema: k-conjuntos
Listar todos os subconjuntos de {1,..., n} de tamanho exatamente k

```
n=5
k=3
conj=[0]*(k+1)

def kconjs(p):
  if p == k + 1:
    print(conj[1:])
  elif p==1:
    for i in range(1,n+1):
      conj[p] = i
      kconjs(p + 1)
  else:
    anterior = conj[p - 1]
    for i in range(anterior + 1,n+1):
      conj[p] = i
      kconjs(p + 1)

kconjs(1)
```

```
def kconjs_iterativo(k,n):
  p = 1
  conj = [0]*(k+1)
  while p>0:
    v = 0
    if conj[p]==0:
      v = conj[p-1]+1
    else:
      v =conj[p]+1
    if v>n:
      conj[p]=0
      p = p-1
    else:
      conj[p]=v
      p = p+1
    if p == k+1:
      print(conj[1:])
      p = p-1
```

p-ésimo elemento

v é o (p-1)-ésimo + 1

v é o valor anterior p/o p-ésimo + 1

candidato v é inválido resetar p-ésimo e voltar para p-1-ésimo

colocar v

achamos

Lista 14A - Q5

Qual o primeiro subconjunto a ser impresso?

Qual o último?

Para $n = 6$ e $k = 3$